

Теория параллелизма

Отчёт Задача №7

Бакумова Валерия Евгеньевна, 23932

г. Новосибирск

2025 г.

Цель: попробовать на практике использование openACC на примере решения уравнения теплопроводности разностной схемой, протестировать ускорение на нескольких ядрах cpu и на gpu.

Компилятор: pgc++ Профилировщик:

Nsight Systems

Замеры времени: `std::chrono::high_resolution_clock` (итоговое время в секундах)

Выполнение на CPU

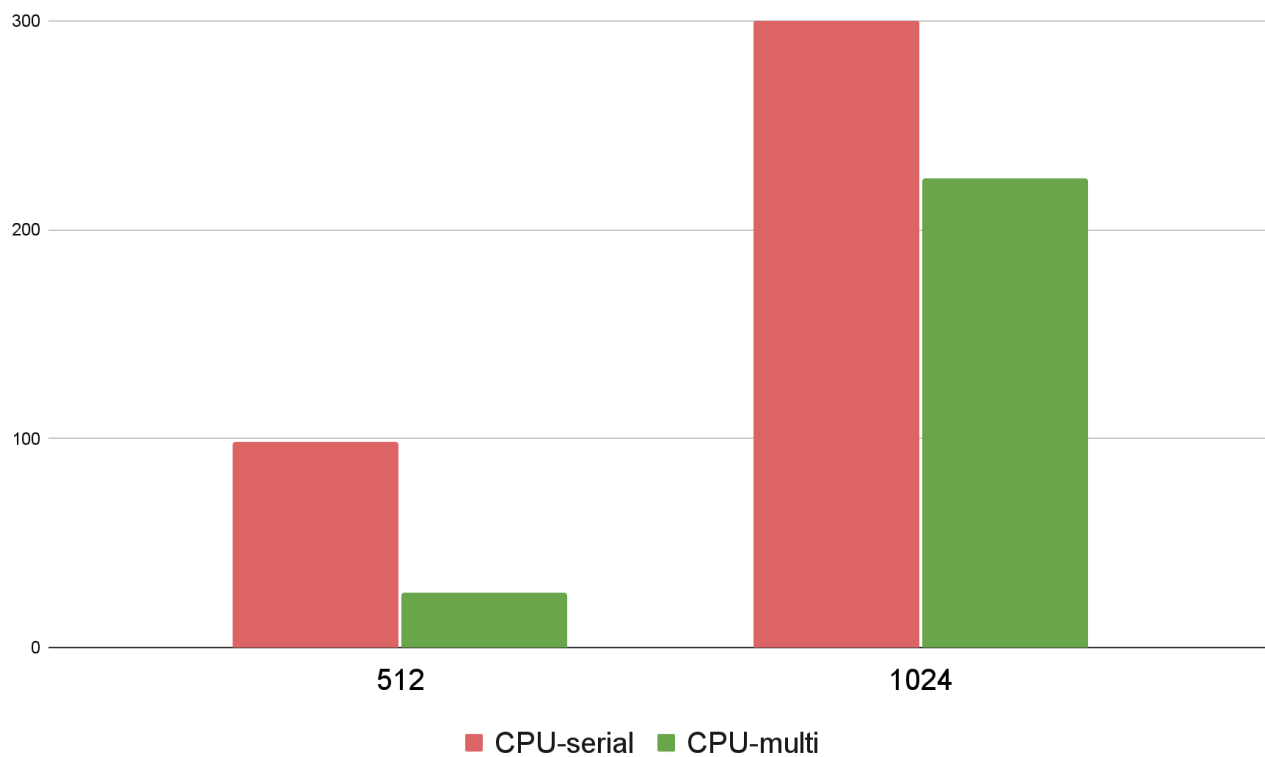
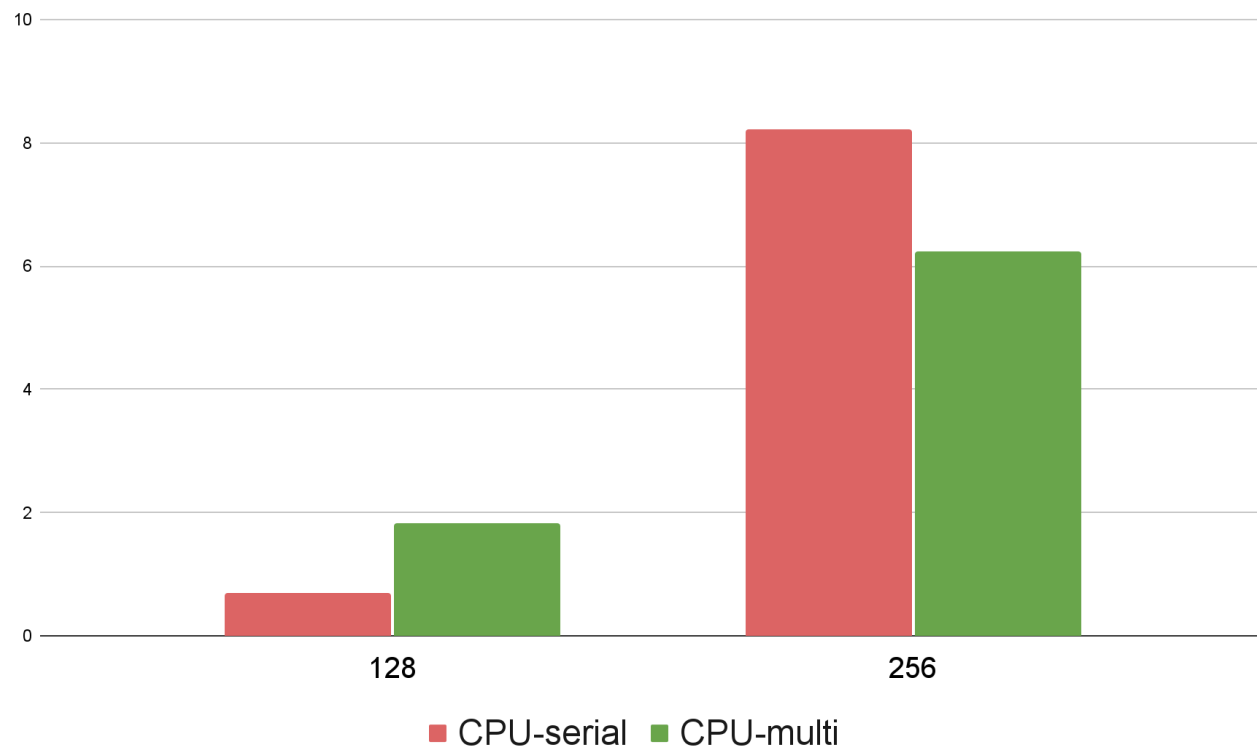
CPU-onecore

Размер сетки	Время выполнения	Ошибка	Количество итераций
128*128	0.703 сек.	4.79656e-08	40000
256*256	8.229 сек.	5.82721e-07	110000
512*512	98.256 сек.	9.92435e-07	340000
1024*1024	>25 мин.	-	1000000

CPU-multicore

Размер сетки	Время выполнения	Ошибка	Количество итераций
128*128	1.84 сек.	4.79656e-08	40000
256*256	6.244 сек.	5.82721e-07	110000
512*512	26.077 сек.	9.92435e-07	340000
1024*1024	224.662 сек.	1.36929e-06	1000000

Диаграмма сравнения CPU-serial и CPU-multi

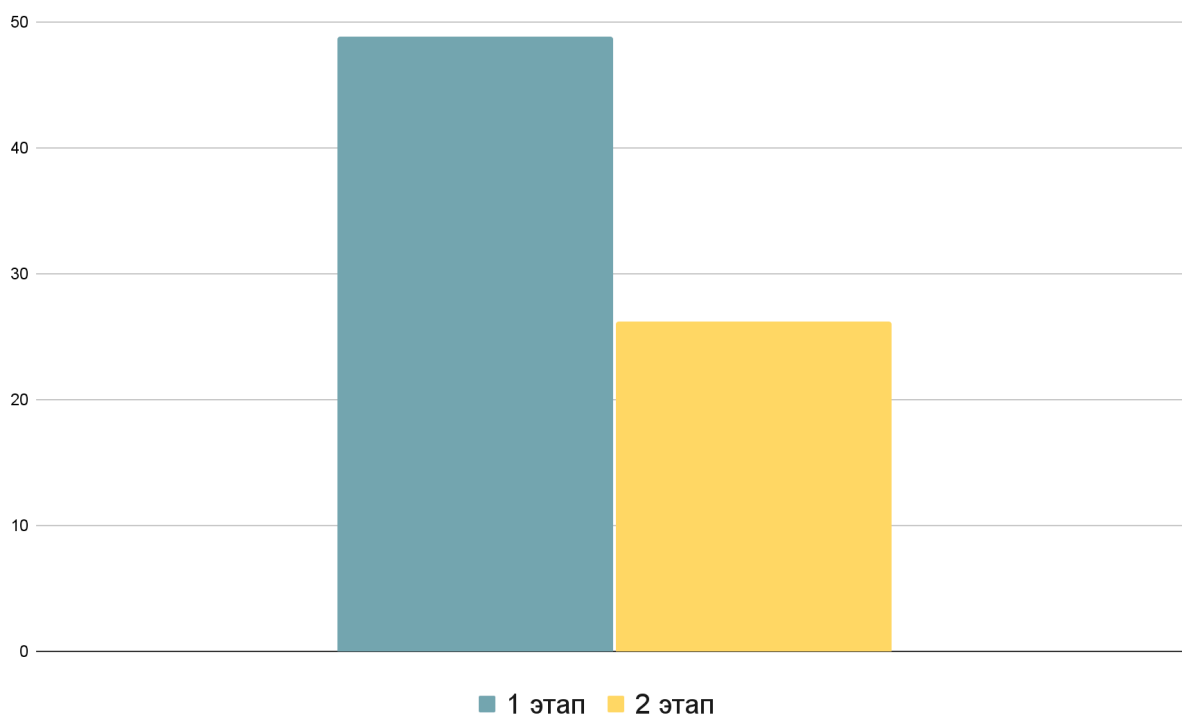


Выполнение на GPU

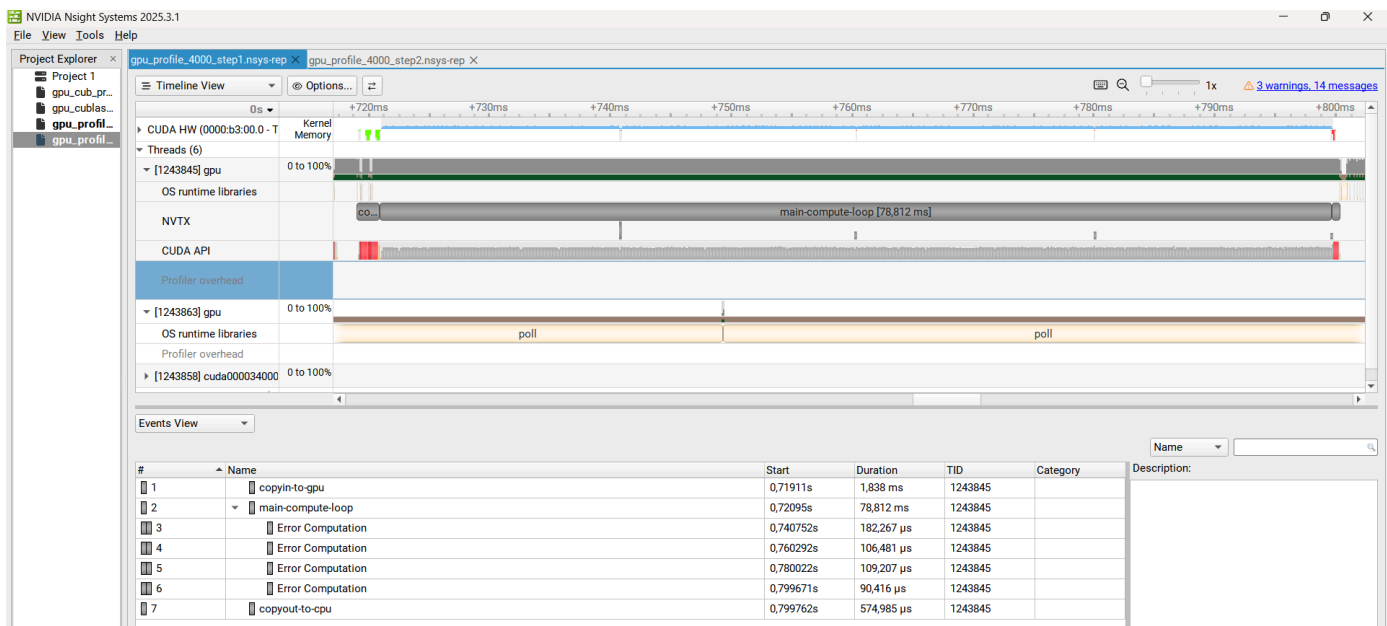
Этапы оптимизации на сетке 512*512

Этап №	Время выполнения	Ошибка	Максимальное количество итераций	Комментарии
1	48.8017 сек.	1.36929e-06	1000000	Базовое применение openACC для ускорения расчётов
2	26.1976 сек.	1.36929e-06	1000000	Использование async при использовании openACC

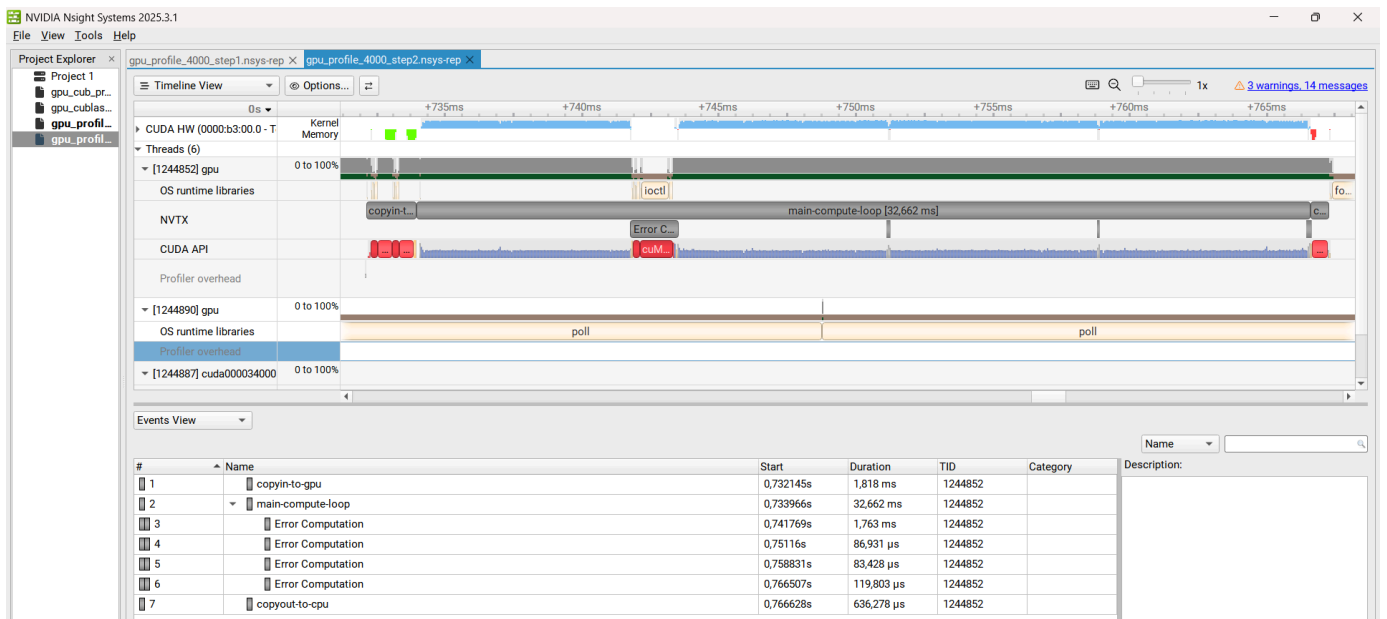
Диаграмма оптимизации



Профилирование этап 1



Профилирование этап 2



GPU - оптимизированный вариант

Размер сетки	Время выполнения	Точность	Количество итераций
128*128	0.110 сек.	4.79656e-08	40000

256*256	0.345 сек.	5.82721e-07	110000
512*512	1.785 сек.	9.92435e-07	340000
1024*1024	25.431 сек.	1.36929e-06	1000000

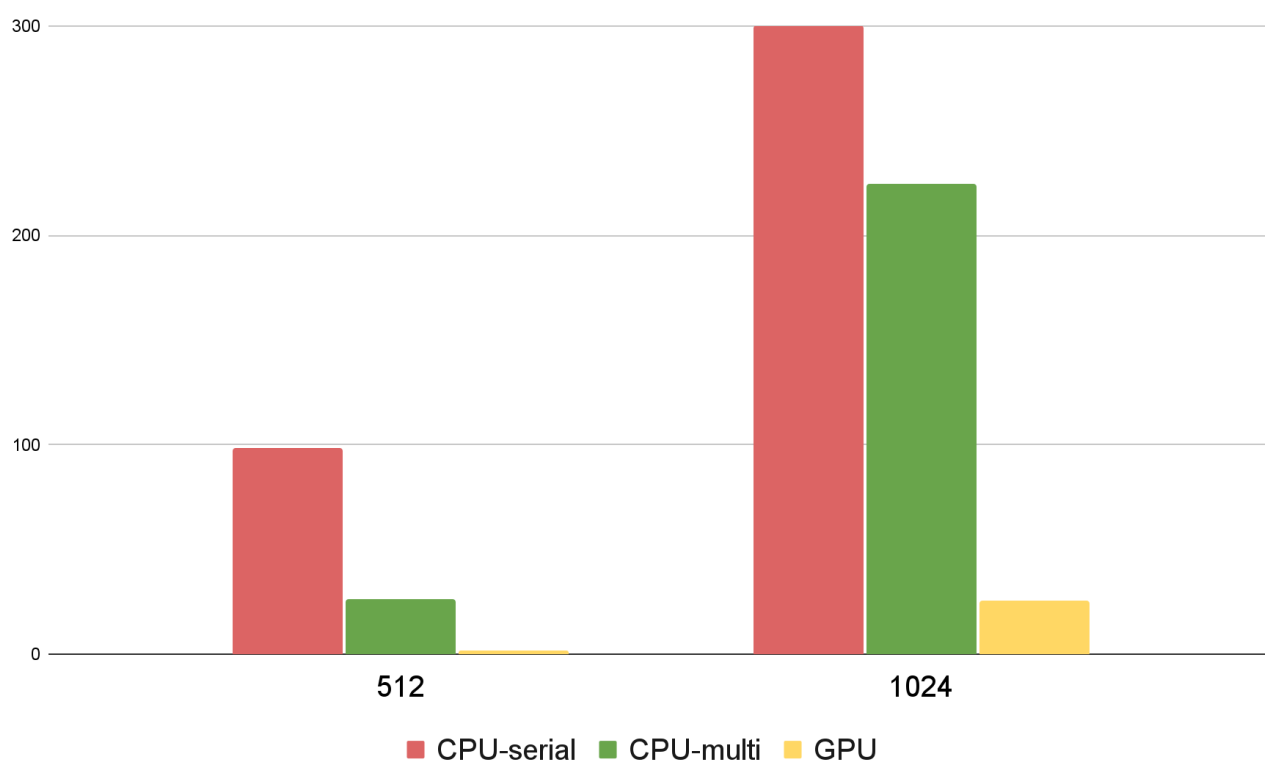
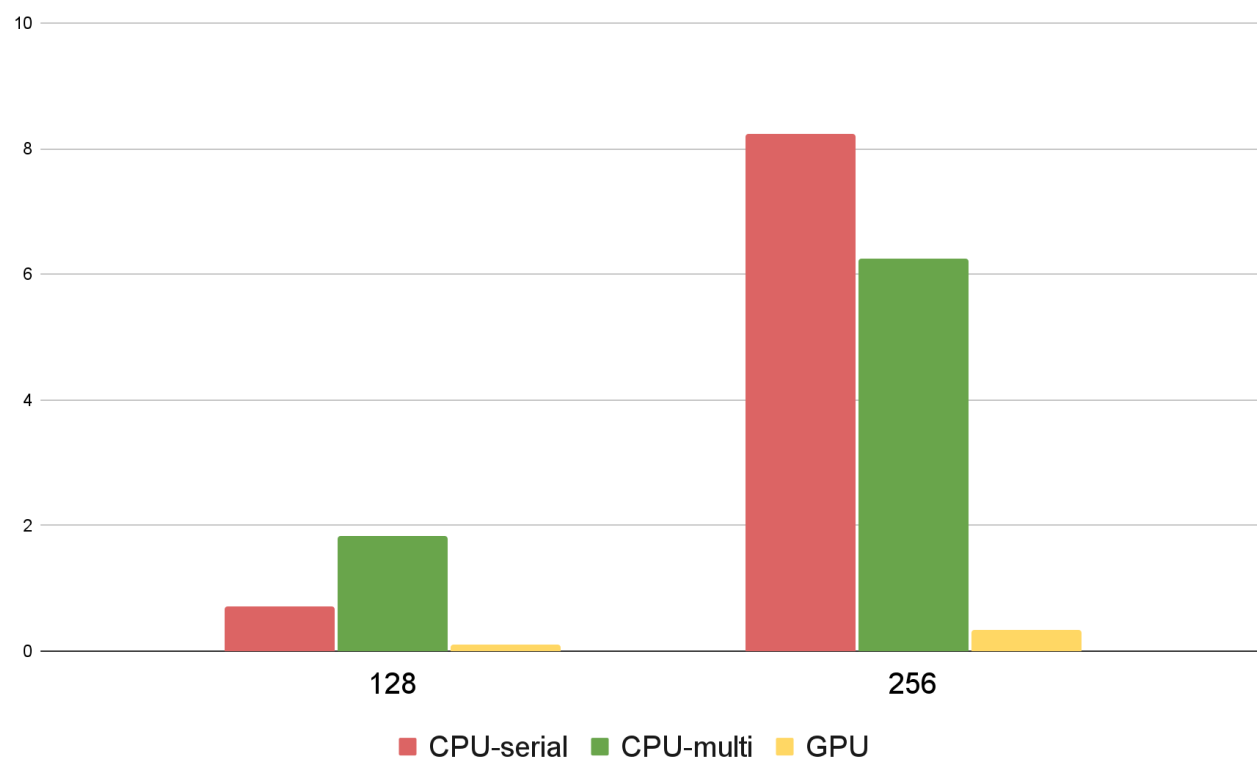
Вывод матрицы 10*10

```

Размер=10
Время: 0.0284797 сек, Ошибка: 0, Итерации: 10000
10 11.1111 12.2222 13.3333 14.4444 15.5556 16.6667 17.7778 18.8889 20
11.1111 12.2222 13.3333 14.4444 15.5556 16.6667 17.7778 18.8889 20 21.1111
12.2222 13.3333 14.4444 15.5556 16.6667 17.7778 18.8889 20 21.1111 22.2222
13.3333 14.4444 15.5556 16.6667 17.7778 18.8889 20 21.1111 22.2222 23.3333
14.4444 15.5556 16.6667 17.7778 18.8889 20 21.1111 22.2222 23.3333 24.4444
15.5556 16.6667 17.7778 18.8889 20 21.1111 22.2222 23.3333 24.4444 25.5556
16.6667 17.7778 18.8889 20 21.1111 22.2222 23.3333 24.4444 25.5556 26.6667
17.7778 18.8889 20 21.1111 22.2222 23.3333 24.4444 25.5556 26.6667 27.7778
18.8889 20 21.1111 22.2222 23.3333 24.4444 25.5556 26.6667 27.7778 28.8889
20 21.1111 22.2222 23.3333 24.4444 25.5556 26.6667 27.7778 28.8889 30

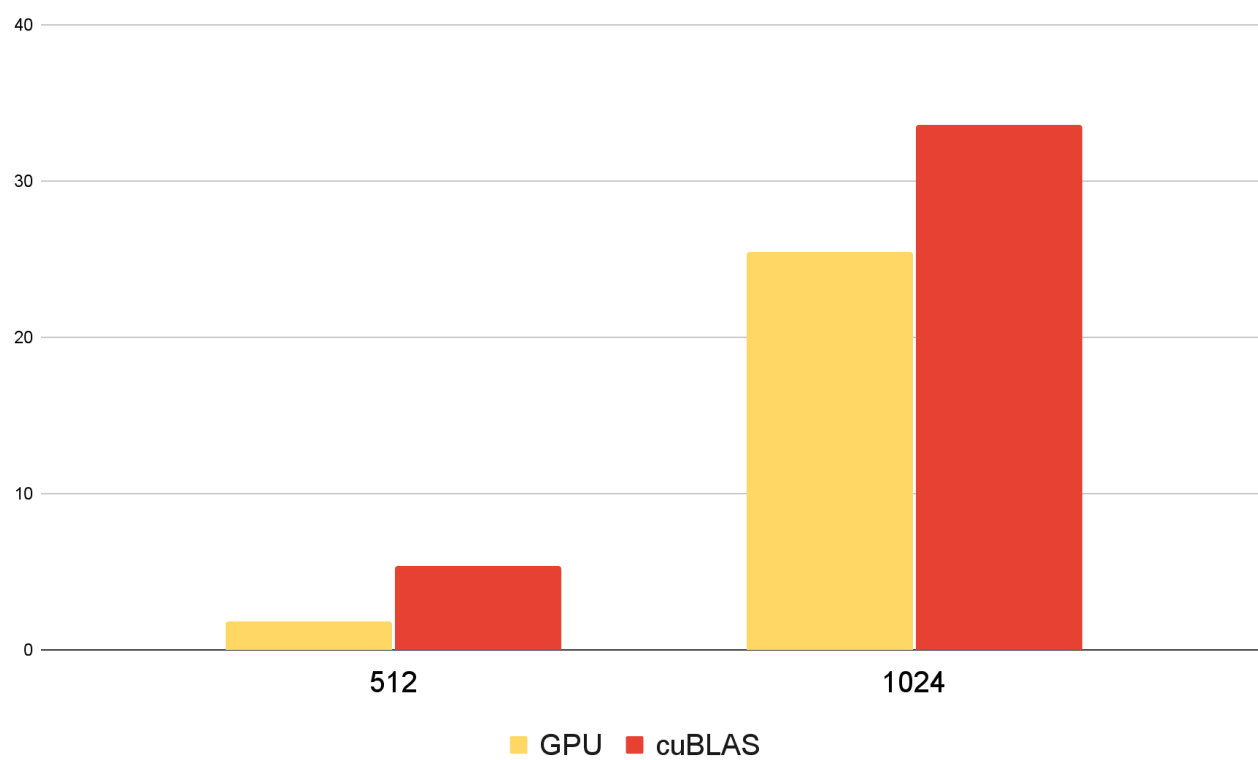
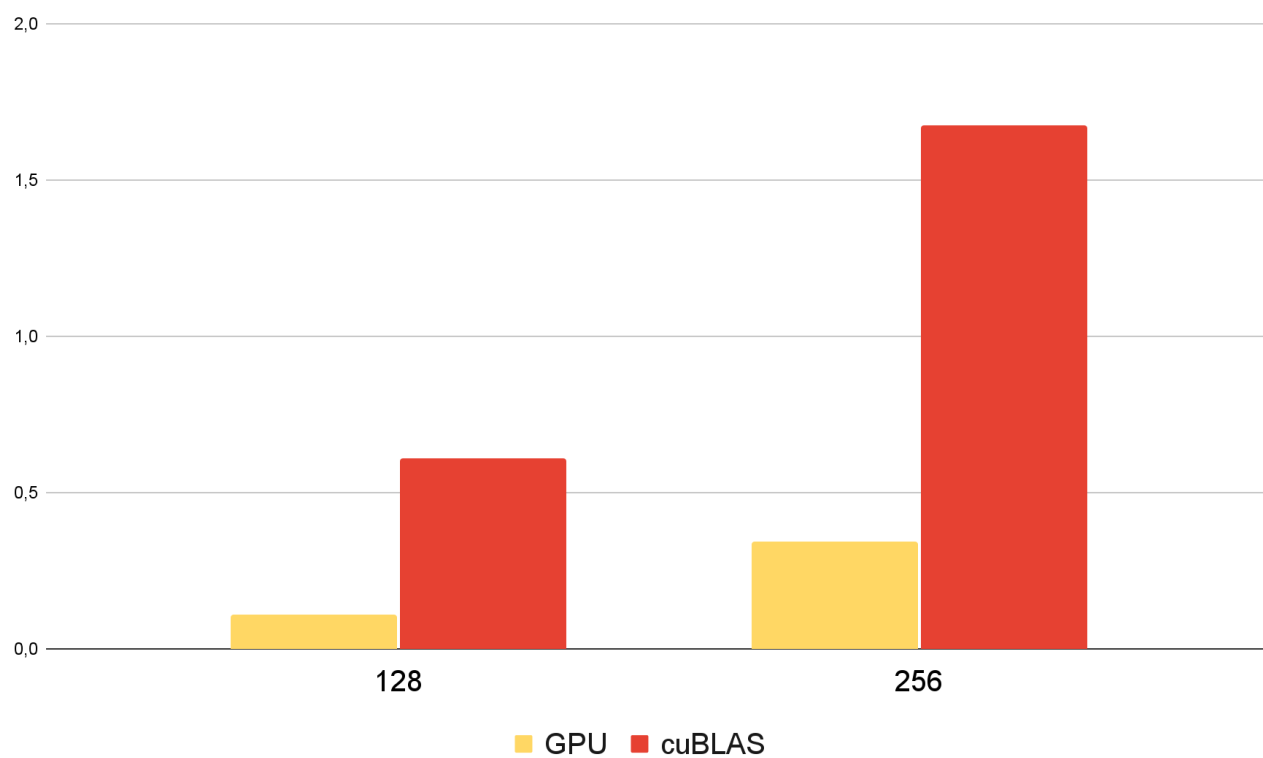
```

Диаграмма сравнения времени работы CPU-serial, CPU-multi, GPU(оптимизированный вариант) для разных размеров сеток



Замена редукции на функции библиотеки cuBLAS

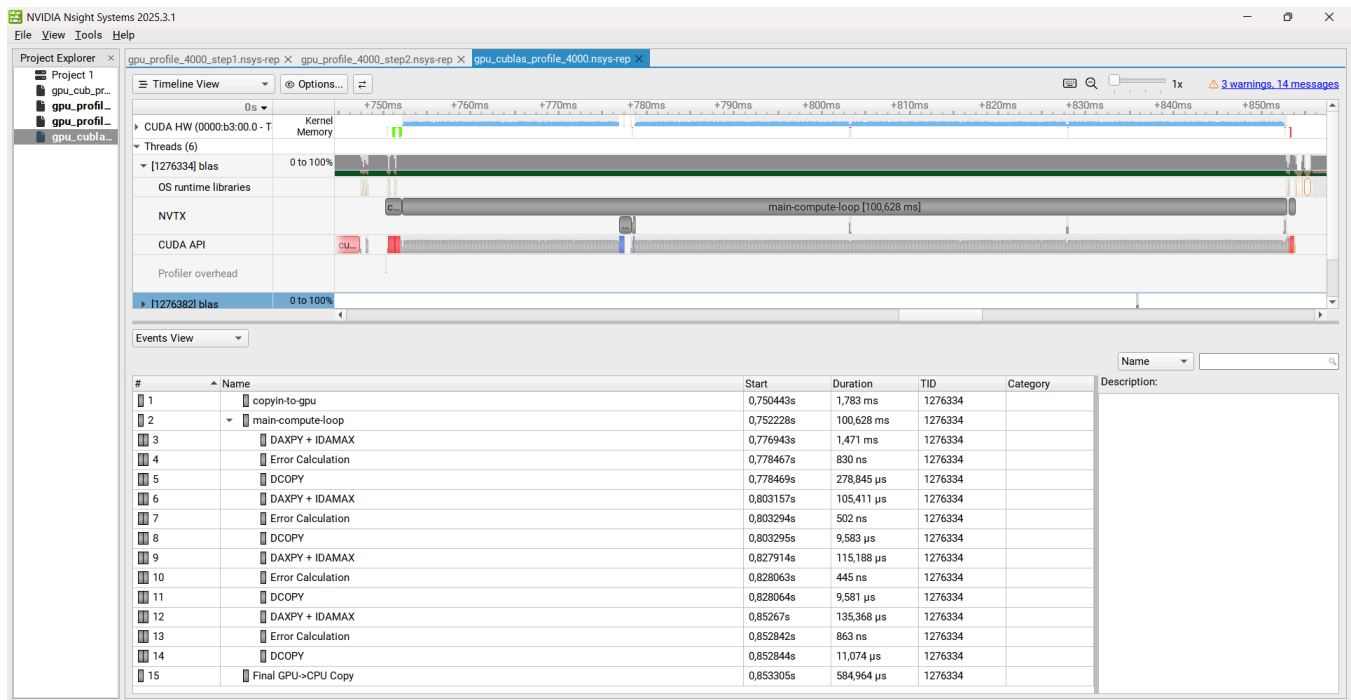
Сравнение программ с cuBLAS и без



cuBLAS реализация

Размер сетки	Время выполнения	Ошибка	Количество итераций
128*128	0.611 сек.	4.79656e-08	40000
256*256	1.677 сек.	5.82721e-07	110000
512*512	5.394 сек.	9.92435e-07	340000
1024*1024	33.620 сек.	1.36929e-06	1000000

Профилирование



Вывод

Как и ожидалось, многопоточный запуск на CPU показывает лучшие результаты по сравнению с одноядерным, а запуск на GPU значительно превосходит многопоточное выполнение. Однако часть производительности GPU теряется из-за затрат на передачу данных между устройством (device) и хостом (host), поэтому такие операции следует минимизировать. Кроме того, операция редукции для вычисления максимальной ошибки (max:error) является достаточно затратной по времени.

Применение функций библиотеки cuBLAS может повысить производительность на больших объемах данных, особенно при полной замене соответствующих участков алгоритма на cuBLAS-оптимизированные версии. В текущей реализации необходимость копирования данных внутри GPU-памяти замедляет вычисление максимальной ошибки, что снижает общий эффект от использования cuBLAS.